

Cloud Infrastructure Planning and Strategy Simulation

Scalable, Cost-Effective and Secure Architecture for a Hypothetical E-Commerce Platform

Prepared as a Week 1 cloud engineering planning simulation

Scenario: high-traffic e-commerce platform | Region: AWS Mumbai | Architecture focus: scalability, security, reliability and cost

1. Executive Summary

This report proposes an AWS-based architecture for a hypothetical e-commerce platform serving approximately 500,000 monthly active users. The design assumes normal traffic of about 1,000 requests per second and campaign peaks of about 5,000 requests per second. The central architectural principle is to keep the application tier stateless so it can scale horizontally while protecting the relational database from avoidable read pressure through caching and asynchronous processing.

The platform uses Route 53, CloudFront, AWS WAF, an Application Load Balancer, a multi-AZ application tier, Amazon RDS for MySQL, ElastiCache for Redis, S3, SQS, CloudWatch, CloudTrail and Secrets Manager. The target is 99.9% application availability with an RTO of 60 minutes or less and an RPO of 15 minutes or less for critical transactional data.

The largest operational risk is not the web tier; it is the database and downstream dependency path during traffic spikes. Therefore, scaling the compute tier alone is insufficient. The design explicitly includes caching, queue-based decoupling, connection limits, health checks, backups and load testing.

2. Scenario and Requirements

The fictional business operates an online storefront selling consumer products. Customers browse product pages, search the catalog, add items to carts, authenticate, place orders, make payments through an external payment provider, and receive order notifications. Traffic is highly variable: normal demand is moderate, while sales campaigns can create short periods of 5× traffic.

Functional requirements include product browsing/search, user accounts, cart management, checkout, order creation, inventory updates, payment integration and notifications. Non-functional requirements include scalability, security, high availability, backup/recovery, observability and controlled cost.

Parameter	Planning assumption
Cloud provider	Amazon Web Services (AWS)
Primary region	Asia Pacific (Mumbai) — ap-south-1
Monthly active users	500,000
Normal traffic	≈1,000 application requests/second
Peak traffic	≈5,000 requests/second during campaigns
Availability target	99.9% for the production application
Initial transactional data	≈500 GB
Object/media storage growth	≈100 GB/month
Application tier	Containerized or EC2-based stateless web/API services
Database	Amazon RDS for MySQL, Multi-AZ
Peak strategy	Auto Scaling; capacity expands based on CPU/request metrics
Recovery objectives	RTO ≤ 60 min; RPO ≤ 15 min for critical transactional data

3. Proposed Architecture

The architecture separates the internet-facing edge from the private application and database layers. Only the required public endpoints are exposed. Application and database workloads run in private subnets, with security groups restricting east-west access.

AWS E-Commerce Reference Architecture — ap-south-1 (Mumbai)

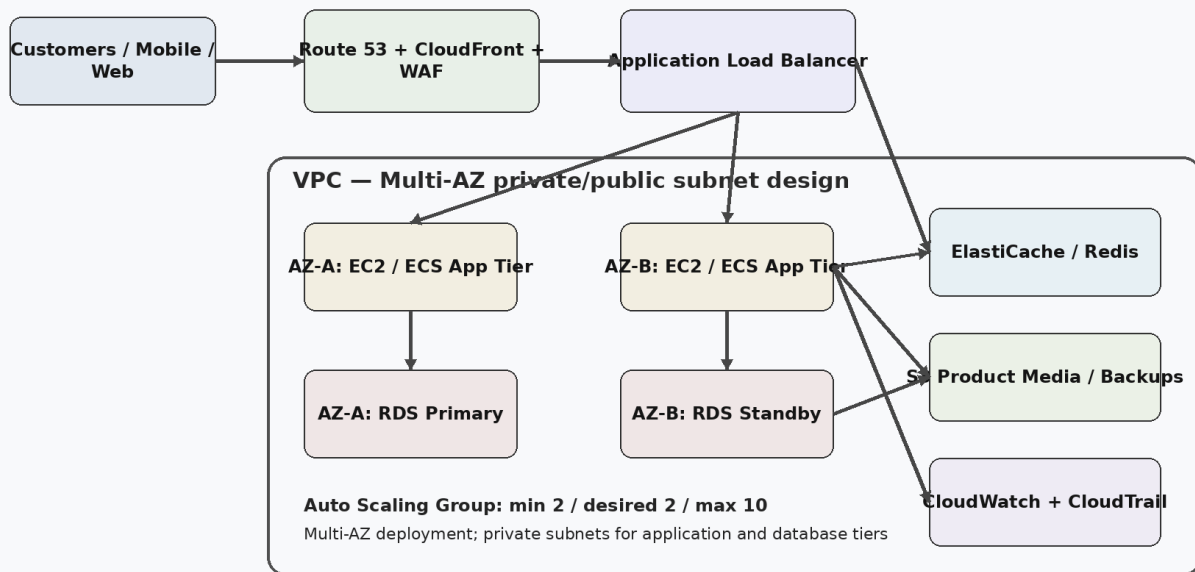
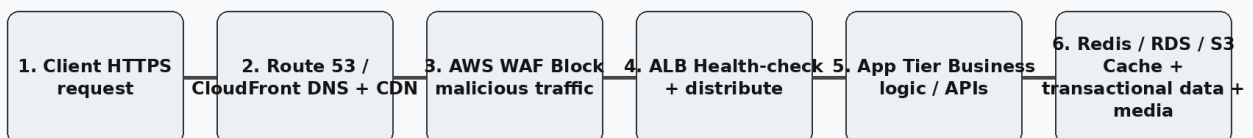


Figure 1. Reference AWS architecture. The application tier is designed for horizontal scaling across at least two Availability Zones. RDS provides the transactional persistence layer, while Redis and S3 reduce unnecessary database and application load.

The production deployment should use Infrastructure as Code (for example, AWS CloudFormation or Terraform) so environments can be reproduced consistently. A CI/CD pipeline should deploy immutable application versions and support rollback.

4. Request and Data Flow

Request and Data Flow



Async workloads: Order events → SQS → background workers → email / inventory / analytics

Observability: CloudWatch metrics + logs + alarms; CloudTrail for API audit; centralized alerts

Recovery: automated RDS backups + snapshots; S3 versioning/lifecycle; cross-region backup copy for critical data

Figure 2. Simplified request and data flow.

Static assets such as product images, CSS and JavaScript should be aggressively cached at CloudFront. Dynamic requests are routed to the ALB and then to healthy application tasks/instances. Frequently requested catalog data can be cached in Redis with a short TTL. Order creation remains transactional in RDS, while non-critical follow-up work such as email, analytics events and some inventory workflows can be moved to SQS-backed workers.

5. Cloud Service Selection and Justification

Layer	AWS service	Purpose / justification
DNS	Amazon Route 53	Highly available DNS and health-check based routing.
CDN	Amazon CloudFront	Caches static content closer to users and reduces origin load.
Security edge	AWS WAF	Filters common web exploits, bots and rate-abuse patterns.
Load balancing	Application Load Balancer	Distributes HTTPS traffic across healthy application instances/tasks.
Compute	EC2 Auto Scaling / ECS	Stateless application tier that scales horizontally with traffic.
Database	Amazon RDS for MySQL	Managed relational database with backups and Multi-AZ failover.
Cache	Amazon ElastiCache for Redis	Reduces repeated database reads for sessions, catalogs and hot data.
Object storage	Amazon S3	Product images, invoices, logs/exports and backup artifacts.
Async messaging	Amazon SQS	Decouples order processing, notifications and background work.
Monitoring	Amazon CloudWatch	Metrics, logs, dashboards and alarms.
Audit	AWS CloudTrail	Records AWS API activity for governance and investigation.
Secrets	AWS Secrets Manager	Stores database/API credentials without hard-coding them.

The service selection favors managed services where the operational burden is otherwise high. RDS, S3, CloudFront, ALB, WAF, CloudWatch and Secrets Manager reduce the amount of infrastructure the team must patch and operate manually. AWS documentation confirms that EC2 is available in Mumbai and that ALB provides managed load balancing with usage-based pricing. ■cite not included in PDF; see references section below.■

6. Scalability Strategy

Horizontal scaling: Application instances/tasks should be stateless and placed behind the ALB. The Auto Scaling Group/ECS service should maintain a minimum of two healthy instances/tasks and scale out when CPU, request count per target or latency crosses defined thresholds.

Traffic absorption: CloudFront handles cacheable content at the edge. WAF applies rate limits and common web protections before requests reach the application.

Database protection: Redis handles hot reads, while application connection pools prevent connection storms. Heavy background work is moved to SQS so a sudden checkout surge does not force every downstream operation to execute synchronously.

Capacity planning: Load testing should validate 5,000 requests/second and at least a safety margin above the assumed peak. Scaling policies should be tuned using measured latency and saturation rather than arbitrary CPU percentages.

7. High Availability and Disaster Recovery

The application tier is deployed across at least two Availability Zones. If one AZ becomes unhealthy, the ALB removes unhealthy targets and traffic is served by the remaining zone. RDS is configured for Multi-AZ deployment so a database failure can trigger managed failover.

Backups should include automated RDS backups with point-in-time recovery, periodic snapshots, S3 versioning for important objects and lifecycle policies for older backup data. For critical business data, backup copies should be replicated to a secondary region. Disaster recovery is incomplete until restores are periodically tested.

Objective	Target	Implementation
Availability	99.9%	Multi-AZ application + RDS, health checks, managed services

Objective	Target	Implementation
RTO	≤ 60 minutes	Automated failover + documented recovery runbook
RPO	≤ 15 minutes	Point-in-time database recovery + replicated critical backups
Backup test	Quarterly minimum	Restore sample database/object set and document result

8. Security Architecture

Security follows least privilege and defense in depth. IAM roles are preferred over long-lived access keys. Administrative access requires MFA. Secrets are stored in Secrets Manager rather than source code or plain-text configuration.

Network controls include separate public/private subnets, restrictive security groups, no direct public database access, controlled outbound internet access and WAF rules at the edge. Data in transit uses HTTPS/TLS. RDS, S3 and backups should use encryption at rest with AWS-managed or customer-managed KMS keys depending on governance requirements.

CloudTrail provides an audit trail of AWS API actions, while CloudWatch collects application and infrastructure telemetry. Alerts should cover high 5xx rates, latency, unhealthy targets, CPU/memory saturation, database storage, database connections and unusual security events.

9. Cost Estimation and Optimization

The following is an academic planning estimate designed to show cost reasoning rather than a billing quote.

Component	Planning basis	Estimated monthly INR
EC2 / application tier	2–6 instances average; burst to 10	■18,000–■30,000
RDS MySQL Multi-AZ	Production DB + storage/backups	■20,000–■35,000
Application Load Balancer	1 ALB + LCU usage	■3,000–■6,000
S3	≈500 GB initial + growth + requests	■1,000–■2,500
CloudFront	CDN traffic and requests	■4,000–■10,000
ElastiCache Redis	Small production cache	■4,000–■8,000
NAT / data transfer	Private subnet egress	■4,000–■10,000
CloudWatch / CloudTrail	Logs, metrics and audit	■2,000–■5,000
SQS / Secrets / Route 53 / WAF	Moderate production usage	■2,000–■5,000
Estimated total	Planning range; excludes tax and unusual spikes	■58,000–■1,11,500 / month

Cost controls: use Auto Scaling instead of permanently running peak capacity; use Savings Plans for predictable baseline compute after usage stabilizes; right-size RDS and cache nodes from monitoring data; use S3 lifecycle policies; control log retention; cache through CloudFront; and create AWS Budgets plus billing alarms.

AWS's published pricing model is usage based. AWS states that EC2 On-Demand is pay-as-you-go and that Savings Plans can provide significant savings for committed usage. ALB pricing is based on running hours plus Load Balancer Capacity Units. The final production estimate should therefore be generated with AWS Pricing Calculator using real traffic and data-transfer measurements.

10. Risk Assessment and Mitigation

Risk	Likelihood	Impact	Mitigation
Traffic spike overload	Medium	High	CloudFront + WAF + ALB + Auto Scaling; load-test before campaigns.
Database bottleneck	Medium	High	Indexes, query tuning, Redis caching, read replicas when justified.
AZ failure	Low	High	Multi-AZ application and RDS deployment with health checks.
Credential compromise	Medium	Critical	IAM least privilege, MFA, Secrets Manager, key rotation, CloudTrail.
DDoS / malicious traffic	Medium	High	CloudFront, AWS WAF, rate limits and AWS DDoS protections.
Data loss/corruption	Low	Critical	Automated backups, point-in-time recovery, versioning and tested restores.
Unexpected cloud bill	Medium	Medium	Budgets, billing alarms, rightsizing, Savings Plans for steady compute.
Deployment failure	Medium	High	CI/CD, immutable releases, health checks and rapid rollback.

11. Implementation Plan

Phase 1 — Foundation: Create the AWS account structure, IAM roles, VPC, subnets, routing, security groups, logging and tagging standards.

Phase 2 — Application platform: Containerize or package the web/API application, deploy it behind the ALB, add health checks and configure Auto Scaling.

Phase 3 — Data layer: Deploy RDS Multi-AZ, S3 buckets, Redis and Secrets Manager. Configure encryption and backups.

Phase 4 — Edge/security: Configure Route 53, CloudFront, WAF rules, TLS certificates and rate limits.

Phase 5 — Observability: Build CloudWatch dashboards and alarms for latency, error rates, saturation, database health and scaling activity.

Phase 6 — Validation: Run load tests at 1,000, 3,000 and 5,000 requests/second. Simulate an AZ failure, database failover, application deployment rollback and restore-from-backup scenario.

Phase 7 — Optimization: Review one month of telemetry, remove unused resources, right-size services and consider Savings Plans for steady compute.

12. Assumptions and Limitations

The traffic, storage, user and recovery figures are assumptions created for the simulation because no real business telemetry was provided. Payment processing is treated as an external managed service; payment card data should not be stored directly unless the organization has a clear compliance architecture.

The design is intentionally more production-oriented than a minimal student demo. For a small proof of concept, the same architecture would be unnecessarily expensive. Conversely, a real global retailer would likely need multi-region traffic management, a more sophisticated data platform, stronger fraud controls and more detailed capacity models.

13. Conclusion

The proposed AWS architecture balances scalability, availability, security and cost by scaling stateless compute horizontally, keeping the database protected behind caching and controlled connections, and using managed services for core infrastructure. The design is capable of absorbing a 5× traffic event without permanently paying for 5× baseline capacity.

The key engineering principle is to validate assumptions with measurements. Before production, the organization should replace the simulated traffic numbers with observed workload data, perform load and failure testing, and update the AWS cost model accordingly.

14. References

1. AWS — Amazon EC2 Pricing and On-Demand Instances documentation.
2. AWS — Elastic Load Balancing Pricing documentation.
3. AWS — Amazon RDS Getting Started Guide and service documentation.
4. AWS — Amazon S3 pricing and storage documentation.
5. AWS — Amazon CloudWatch, CloudTrail, WAF, CloudFront, Route 53, SQS, ElastiCache and Secrets Manager documentation.

Pricing pages were reviewed for this planning exercise in September 2026. Exact prices vary by AWS region, usage and pricing model; therefore the INR estimates above should be treated as academic planning ranges, not quotations.